PROJECT :: Threaded HTTP Server in Java

- Due Nov 24 by 11:59pm
- Points 250
- Submitting a file upload
- File Types java
- Available until Nov 26 at 11:59pm

This assignment was locked Nov 26 at 11:59pm.

Programming Assignment: Building a Multi-Threaded Web Server

Overview

In this assignment, you will develop a multi-threaded web server in Java capable of processing multiple simultaneous HTTP and HTTPS requests in parallel. Your completed server will handle basic HTTP responses, serve files from a designated directory, and support secure communication using SSL/TLS. This project will guide you through implementing core web server functionalities, handling HTTP GET requests, managing errors, and integrating HTTPS for secure connections. By completing this assignment, you will deepen your understanding of HTTP and HTTPS protocols, Java I/O operations, multi-threading, and the fundamental principles of web server security and functionality.

Project Phases

This project will be completed in a single iteration, but it will be broken down into three major parts to guide your development:

1. **Part 1: Basic Web Server Implementation**
    - Implement a web server that listens on a specified port (e.g., 6789).
    - Handle incoming TCP connections in a multi-threaded manner, with each request processed in a separate thread.
    - Display the incoming HTTP request content in the console for inspection.
    - Send back a basic HTTP response with plain text or simple HTML.
    - Close the input/output streams and the connection socket after processing each request.
2. **Part 2: Handling GET Requests and File Serving**
    - Enhance the server to handle GET requests for static files.
    - Extract the requested file name from the HTTP request line and serve the file if it exists within the current directory.
    - Implement basic error handling:
        - Respond with a `404 Not Found` status and an error page if the requested file does not exist.
        - Send a `405 Method Not Allowed` status if the request method is not supported.

- Support common MIME types (e.g., `text/html`, `image/jpeg`, `image/gif`).
- Include a mechanism to display headers from the client's HTTP request for learning purposes.

3. **Part 3: Implementing HTTPS**
   - Introduction to HTTPS in Java
   - Setting up a keystore
   - Configuring the server for HTTPS
   - Running HTTP and HTTPS concurrently
   - Permissions and port considerations
   - Testing and verification

Requirements

Multi-threaded Design: Your server must handle multiple simultaneous client connections by creating a new thread for each incoming request, ensuring that it can process requests in parallel without blocking.

Port Configuration: Begin by configuring your server to listen on a port higher than 1024 (e.g., 6789 or 8080) for both HTTP and HTTPS. This will simplify initial testing and development by avoiding permission issues associated with privileged ports. Include the port number in the URL when accessing the server from a browser (e.g., `http://localhost:8080/index.html`). Once basic functionality is confirmed, you can modify the configuration to use standard ports (80 for HTTP and 443 for HTTPS), which may require elevated permissions.

File Serving: Implement file serving for GET requests to deliver files relative to the server's running directory. Ensure the server validates and sanitizes the requested paths to prevent directory traversal vulnerabilities, maintaining safe file access.

> Sanitization ensures the server is protected against directory traversal attacks, where malicious users attempt to access files outside the intended directory using relative paths like `../`. For example, a request such as `GET /../../etc/passwd` could expose sensitive files if proper precautions are not taken. To prevent this, the server should restrict file paths to the running directory by combining the requested path with the server's base directory and resolving the result to its canonical form using methods like `File.getCanonicalPath()`. This process eliminates any relative path components like `../`.
>
> After resolving the path, it is essential to validate that the resolved path still resides within the server's designated directory. If it does not, the request should be rejected. Normalizing the input is also necessary to decode URL-encoded characters, such as `%2e%2e/`, and to remove redundant path components like multiple slashes (`//`). This ensures the path is sanitized before file access.
>
> If a path points outside the allowed directory, the server should respond with an appropriate error, such as `403 Forbidden` or `404 Not Found`. By carefully sanitizing paths and validating access, the server avoids unauthorized file exposure or unintended access to sensitive parts of the file system.

HTTPS Integration: Configure the server to support HTTPS connections using an SSL/TLS certificate stored in a keystore. Begin testing HTTPS on a high port number (e.g., 8443) before transitioning to port 443 for real-world simulation and secure communication.

Error Handling: Implement error handling to provide meaningful responses:

- Return a 404 Not Found status and an error page if the requested file does not exist.
- Return a 405 Method Not Allowed status for unsupported HTTP methods.

Secure Communication: Ensure the server can support both HTTP and HTTPS traffic concurrently. Properly configure and test SSL/TLS to handle secure data transmission and prevent vulnerabilities associated with unencrypted connections.

Part 1: Basic Web Server Implementation

Introduction: In this first part of the assignment, you will establish a foundational understanding of how a basic web server functions by developing a multi-threaded server that processes incoming HTTP requests and sends simple responses. The focus is on building the essential structure for handling client connections and generating basic HTTP responses. More advanced features, such as serving specific files and handling various HTTP methods, will be covered in Part 2.

Implementation Details

Main Structure:

- Create a main class, `WebServer`, that initializes the server and listens for incoming client connections.
- Develop an `HttpRequest` class that implements `Runnable` to handle each client request in a separate thread, allowing concurrent processing of multiple connections.

Basic Request Handling:

- Read and log the incoming HTTP request content to the console for inspection.
- Construct and send a basic HTTP response, such as a "Hello, World!" message, with a simple "200 OK" status.

Stream Management:

- Use Java's `InputStream` and `OutputStream` to handle basic data flow between the client and server.
- Incorporate `BufferedReader` for reading incoming request data and `DataOutputStream` for sending responses back to the client.
- Ensure proper closing of input/output streams and the connection socket after processing each request to maintain resource efficiency.

Example Workflow:

1. The server runs on port 6789, waiting for client connections.
2. A client (e.g., web browser) sends a request, such as `GET /`.
3. The server logs the received request details to the console for visibility.
4. The server responds with a simple HTTP response:
   - Status line: `HTTP/1.1 200 OK`
   - Basic body content: "Hello, World!"
5. The server closes the input/output streams and the client socket after the response is sent, completing the request-handling cycle.

HTTP Server Scaffolding

This scaffolding provides a minimal starting point for your HTTP server project. The provided `HTTPServer` and `HttpRequestHandler` classes form the foundation of a simple, multi-threaded web server that listens for HTTP connections and responds with a "Hello, World!" message to any request. This basic server demonstrates core concepts like handling socket connections, reading HTTP request lines, and sending responses.

Purpose and Structure

- **Functionality**: The provided server is fully functional but limited. It runs on a specified port (e.g., 6789) and handles incoming requests in separate threads, illustrating the basics of multi-threading and HTTP request handling.
- **Learning Objective**: This scaffolding is designed to help you understand how web servers process HTTP requests, manage client connections, and structure responses.

Instructions for Students

- **Do Not Change Class Names**: Maintain the class names `HTTPServer` and `HttpRequestHandler` as provided. This ensures uniformity in submissions and ease of grading.
- **Start Here**: Use this scaffolding as your starting point. Extend the code to meet additional requirements, such as serving specific files, handling different HTTP methods, and improving response handling.
- **Focus on Enhancements**: Your main task is to build upon this basic structure, adding features that align with the project guidelines, such as custom responses, file serving, and error handling.

Begin by reviewing the code to understand how it processes incoming connections and sends a basic response. Then, incrementally add functionality as specified in your assignment.

```
import java.io.*;
import java.net.*;

public class HTTPServer {
    public static void main(String[] args) {
        // Set the port number
        int port = 6789;

        try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("HTTP Server running on port " + port);
```

```
            // Infinite loop to listen for incoming connections
            while (true) {
                Socket clientSocket = serverSocket.accept();

                // Create a new thread to handle the request
                HttpRequestHandler requestHandler = new HttpRequestHandler(clientSocket);
                Thread thread = new Thread(requestHandler);
                thread.start();
            }
        } catch (IOException e) {
            System.err.println("Server exception: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

class HttpRequestHandler implements Runnable {
    private Socket socket;

    public HttpRequestHandler(Socket socket) {
        this.socket = socket;
    }

    @Override
    public void run() {
        try (InputStream is = socket.getInputStream();
             DataOutputStream os = new DataOutputStream(socket.getOutputStream());
             BufferedReader br = new BufferedReader(new InputStreamReader(is))) {

            // Read the request line
            String requestLine = br.readLine();
            System.out.println("Received request: " + requestLine);

            // Send a basic HTTP response
            String response = "HTTP/1.1 200 OK\r\n" +
                              "Content-Type: text/plain\r\n" +
                              "\r\n" +
                              "Hello, World!";
            os.writeBytes(response);

        } catch (IOException e) {
            System.err.println("Request handling exception: " + e.getMessage());
            e.printStackTrace();
        } finally {
            try {
                socket.close();
            } catch (IOException e) {
                System.err.println("Socket closing exception: " + e.getMessage());
            }
        }
    }
}
```

Part 2: Handling GET Requests and File Serving

Introduction: In Part 2, you will extend the functionality of your basic web server to handle GET requests for serving static files. This part builds on the foundational multi-threaded server from Part 1 by introducing more complex features, such as parsing HTTP requests, serving files from the server's directory, and implementing proper response formatting. Additionally, you will implement error handling to manage common issues like missing files and unsupported HTTP methods. This phase will deepen

your understanding of HTTP request parsing, file I/O operations, and how to construct detailed HTTP responses with appropriate status codes and headers.

Implementation Details

Enhancing Request Processing:

- Extend the `HttpRequestHandler` class to parse and handle GET requests from the incoming HTTP request line.
- Extract the requested file path from the HTTP request line and ensure safe access by sanitizing the input to prevent directory traversal vulnerabilities (e.g., `../`).

File I/O and Response Handling:

- Implement file reading functionality to serve static files from the server's running directory.
- If the requested file exists, send a 200 OK status with the file's content and the appropriate `Content-Type` header.
- Use Java's `FileInputStream` and `BufferedReader` for reading files, and `DataOutputStream` for sending the response.
- Create a helper method `contentType()` to determine the MIME type of the requested file based on its extension (e.g., `.html`, `.jpeg`, `.gif`).

Error Handling:

- Implement error handling to manage cases where:
  - The requested file does not exist: respond with a 404 Not Found status and a simple error page.
  - The HTTP method is not GET: respond with a 405 Method Not Allowed status.
- Create basic error response pages in HTML to be sent in the event of a 404 or 405 response.

Response Formatting:

- Construct and send an HTTP response with the status line, headers, and body.
- Use CRLF (`\r\n`) to separate headers and body content according to the HTTP standard.

Example Workflow

1. **Server Workflow**:
   - The server runs on port 6789 and listens for incoming connections.
   - A client (e.g., web browser) makes a request such as `GET /index.html`.
2. **Request Processing**:
   - The server logs the request details and extracts the requested file path from the HTTP request line.
   - The server checks if `index.html` exists in the running directory:
     - **If found**: The server responds with a `200 OK` status and serves the file content, including the correct `Content-Type` header.

- **If not found**: The server responds with a `404 Not Found` status and an error page.
- **If the request method is unsupported**: The server responds with a `405 Method Not Allowed` status.

3. **Response**:
   - The server sends the response headers, followed by the file content or error message.
   - The input/output streams and the connection socket are closed after the response is sent.

Part 3: Implementing HTTPS Support

Introduction: In Part 3, you will enhance your web server by adding support for HTTPS, enabling secure communication through SSL/TLS. This section builds on the HTTP functionality developed in earlier parts and introduces encryption and secure data transmission. You will learn how to set up a keystore, generate a self-signed SSL/TLS certificate, configure the server to use the certificate, and handle both HTTP and HTTPS traffic concurrently. Additionally, this part provides practical guidance for testing and validating secure connections.

How HTTPS Works in Java

Enabling HTTPS in Java involves using an `SSLServerSocket`, a secure version of `ServerSocket` capable of handling encrypted connections. HTTPS relies on a certificate that contains a public key, which is used by clients to verify the server's identity during the SSL/TLS handshake. This certificate, along with its associated private key, is stored in a keystore. For local development, you will generate a self-signed certificate and store it in a keystore file (e.g., `keystore.jks`).

Generating a Self-Signed Certificate

To generate a self-signed certificate for development, use the `keytool` utility, which is included with Java. The following command creates a keystore containing the certificate and private key:

```
keytool -genkeypair -alias serverkey -keyalg RSA -keystore keystore.jks -keysize 2048
```

During this process, you will be prompted to provide details such as the keystore password, the distinguished name for the certificate, and other information. These details will appear in the certificate presented to clients.

Exporting the Public Certificate

After creating the keystore, you need to export the public certificate so it can be imported into browsers or testing tools. Use the following command to export the certificate:

```
keytool -export -alias serverkey -file server.crt -keystore keystore.jks
```

This command generates a `.crt` file containing the public certificate. This file will be used to add your certificate to your browser’s trusted certificate store.

Configuring HTTPS in Your Server

To enable HTTPS in your server, configure Java to recognize and access the keystore. Set the following system properties in your server code to specify the keystore file and password:

```
System.setProperty("javax.net.ssl.keyStore", "keystore.jks");
System.setProperty("javax.net.ssl.keyStorePassword", "yourKeystorePassword");
```

Next, use an `SSLServerSocketFactory` to create an `SSLServerSocket` that listens for HTTPS connections. This socket should run on a separate port (e.g., 443 for production or 8443 during development). For instance:

```
SSLServerSocketFactory sslServerSocketFactory =
    (SSLServerSocketFactory) SSLServerSocketFactory.getDefault();
SSLServerSocket sslServerSocket =
    (SSLServerSocket) sslServerSocketFactory.createServerSocket(8443);
```

You should implement this socket in a separate thread to handle HTTPS traffic concurrently with the HTTP server.

Requirements:

1. **Use Standard Ports**:
   - HTTP should run on port 80.
   - HTTPS should run on port 443.
   - Ensure that your server can listen on both ports simultaneously.
2. **Generate an SSL/TLS Certificate**:

   ```
   keytool -genkeypair -alias serverkey -keyalg RSA -keystore keystore.jks -keysize 2048
   ```

   Use `keytool` to create a self-signed certificate stored in a keystore. Remember to keep track of your keystore file and password.

3. **Configure the HTTPS Server**:

   ```
   System.setProperty("javax.net.ssl.keyStore", "keystore.jks");
   System.setProperty("javax.net.ssl.keyStorePassword", "yourKeystorePassword");
   ```

   Load the keystore and set up the `SSLServerSocket`. Implement a separate `SSLServerSocket` that runs alongside the `ServerSocket` for HTTP.

4. **Testing**:
   - Verify that your server responds to both `http://localhost` and `https://localhost` without specifying port numbers, since the server will use the standard ports.
   - Use a browser or tools like `curl` to confirm that HTTPS connections are encrypted.

5. **Keystore and HTTPS Configuration**

To enable HTTPS, your server must accept the keystore file and password as command-line arguments. If these arguments are missing, your server should gracefully fall back to HTTP-only mode, allowing it to function without SSL/TLS. This behavior ensures that the server remains usable for testing and development, even when a keystore is unavailable.

1. **Command-Line Arguments**:
   - Your server must accept two optional arguments:
     - The path to the keystore file (e.g., `keystore.jks`).
     - The keystore password.
   - Example:
     ```
     java WebServer.java keystore.jks keystorePassword
     ```
2. **Fallback Behavior**:
   - If no keystore arguments are provided, the server should start in HTTP-only mode on port 80.
   - If both arguments are provided, the server must start HTTP on port 80 and HTTPS on port 443.
3. **Error Handling**:
   - If only one argument is provided, log an error to stderr and default to HTTP-only mode.

Submission

Submit your Java file with a single class structure, with all relevant code for both parts of the project in one file. Ensure the file is formatted as a Java 11 Single Source Executable so that running `java WebServer.java` will compile and run the server.

Version History

V003 : Updated http/https operation and how you pass in the keystore and keystore password
V002 : Updated part 3 for greater clarity. Updated section on sanitization.
V001 : Expect bugs.